



Backend Development Mastery with Server Side Rendering with Django & PostgreSQL

Full Backend Development with Server Side Rendering (Frontend) concepts and implementation using Django & Postgresql.

1. Total course length : 90-100 hours
2. Tech Stack: Python 3.11+, Django 5.x, PostgreSQL 14+, Django REST framework, Git, Docker, HTML, CSS, Javascript.
3. Each classes will be having an assignment. Tracked and Evaluated in Google classroom.
4. Students will be graded by quiz, assignments and projects.

Chapter 00 - Basics of Python Project Oriented

1. Introduction to Python

- What is Python and its key features
- Installing Python and setting up the environment
- Running Python scripts
- Python interactive shell (REPL)
- Writing your first Python program

2. Python Basics

- Variables and data types
 - int, float, str, bool
 - Type conversion
- Basic input/output
 - input() function
 - print() function and formatting
- Comments and code readability

3. Operators

- Arithmetic operators
- Comparison operators
- Logical operators
- Assignment operators
- Membership operators (in, not in)
- Identity operators (is, is not)

4. Control Flow

- if, elif, and else statements
- Nested conditions
- match statement (Python 3.10+)

5. Loops

- for loops
- while loops
- break, continue, and pass
- Iterating over lists, strings, and ranges

6. Data Structures

- Lists
 - Creating and modifying lists
 - List slicing
 - Common list methods
- Tuples
 - Immutable nature
 - Tuple unpacking
- Dictionaries
 - Key-value pairs
 - Accessing, adding, and removing items
- Sets
 - Unique elements
 - Set operations (union, intersection, difference)

7. Functions

- Defining and calling functions
- Parameters and return values
- Default arguments
- Keyword arguments
- Variable-length arguments (*args, **kwargs)
- Scope (local, global, and nonlocal)

8. Strings in Depth

- String indexing and slicing
- String methods
- f-strings and formatting

9. Modules and Packages

- Importing built-in modules
- Writing your own modules
- Understanding `__name__ == "__main__"`

10. File Handling

- Reading and writing text files
- Working with file paths
- Handling exceptions in file operations

11. Exception Handling

- try, except blocks
- finally clause
- Raising exceptions
- Custom exceptions

12. Introduction to Object-Oriented Programming (OOP)

- Classes and objects
- Instance variables and methods
- Constructors (`__init__`)
- Basic inheritance
- Method overriding

13. Working with Virtual Environments

- Why use virtual environments
- Creating and activating venv
- Installing packages with pip

14. Pythonic Practices

- PEP 8 guidelines
- Writing clean, readable code
- Using list comprehensions and dictionary comprehensions

15. Projects

- Build a game where the user guesses a randomly generated number.
- Build a calculator application.
- Build a calculator application as a game
- Build a command line todo application
- Build a simple version of Hangman
- Build a Caesar Cipher: Implement a basic encryption and decryption tool
- Build a simple pomodoro timer
- Build a simple key-value storage system
- Build a simple grocery list

Chapter 1: Internet & Networking Fundamentals

1.1 How the Internet Works

- Packets, routing, IP addresses (IPv4 vs IPv6)
- Role of ISPs
- Basics of latency, bandwidth, and throughput

1.2 Domain Name System (DNS)

- Domain resolution process
- A, AAAA, CNAME, MX, TXT records
- DNS caching and propagation

1.3 HTTP & HTTPS Protocols

- Request/Response cycle
- HTTP methods: GET, POST, PUT, PATCH, DELETE
- Status codes and their categories
- HTTPS, TLS/SSL handshake

1.4 Client-Server Architecture

- Frontend vs backend responsibilities
- APIs and their communication

1.5 Tools & Testing Basics

- Using curl, Bruno, and browser dev tools

Chapter 2: Python Refresher for Web Development

- Functions, classes, modules
- Decorators, context managers
- Virtual environments and dependency management (pip, venv, pip-tools)

Chapter 3: Django Fundamentals

3.1 Introduction to Django

- What is Django, MVT architecture
- Comparison with Flask, FastAPI

3.2 Project Setup

- Creating Django project and app
- Project structure walkthrough

3.3 URL Routing & Views

- Function-based views (FBVs)
- Class-based views (CBVs)
- URL patterns and parameters

3.4 Templates

- Django template language (DTL)
- Template inheritance

3.5 Static & Media Files

- Configuring static and media files
- Serving in development and production

3.6 Forms & Validation

- Django forms vs ModelForms
- Form validation and error handling

3.7 User Authentication

- Django's built-in auth system
- Login, logout, password reset

Chapter 4: Django with PostgreSQL

4.1 PostgreSQL Fundamentals

- Installing PostgreSQL
- Basic SQL commands (CRUD)
- Primary keys, foreign keys, indexes

4.2 Django ORM (4 hours)

- Models and migrations
- QuerySets (filter, exclude, annotate, aggregate)
- Relationships: One-to-One, One-to-Many, Many-to-Many

4.3 Advanced ORM Usage

- Raw SQL queries
- Database transactions

4.4 Performance Optimization

- Indexing strategies
- Query optimization

Chapter 5: Django Internals & Advanced Features

- Middleware deep dive
- Signals and their use cases
- Context processors
- Custom management commands
- Working with environment variables

Chapter 6: APIs with Django REST Framework

- Setting up DRF
- Serializers (basic, nested, ModelSerializer)
- ViewSets and Routers
- Authentication and permissions
- Pagination, filtering, and ordering

Chapter 7: Deployment & Production Practices

- Settings for production
- Serving static files with WhiteNoise
- Using Gunicorn & Nginx
- Database migrations in production
- Environment variable management

Chapter 8: Capstone Project

- Plan, design, and build a backend for a real-world app
- Examples: Blog platform, Task manager, Inventory system
- Integrating PostgreSQL, DRF, and authentication
- Deployment to a cloud platform (e.g., DigitalOcean, AWS)

Learning Outcomes

1. Strong Backend Foundations
 - Understand core backend concepts like HTTP, APIs, authentication, and data modeling.
 - Learn how servers process requests and deliver responses efficiently.
2. Proficiency in a Powerful Web Framework
 - Master Django's features: ORM, admin interface, middleware, and template system.
 - Write clean, maintainable, and scalable backend code.
3. Advanced Database Skills
 - Gain hands-on expertise in PostgreSQL, including indexing, joins, triggers, and views.
 - Optimize queries for speed and handle large-scale datasets.
4. API Development Mastery
 - Build RESTful and GraphQL APIs from scratch.
 - Implement pagination, filtering, and versioning for real-world use cases.
5. Authentication & Authorization Expertise

- Implement secure user login systems with sessions, JWT, or OAuth.
 - Manage role-based permissions for different types of users.
6. Real-World Problem Solving
 - Work on projects like social media platforms, e-commerce backends, or analytics dashboards.
 - Learn how to debug, log, and handle production issues.
 7. Deployment & Scaling Skills
 - Deploy Django apps to cloud platforms like AWS, Azure, or DigitalOcean.
 - Learn load balancing, caching, and database scaling techniques.
 8. Performance Optimization
 - Use query optimization, caching strategies, and async views to boost performance.
 - Monitor and improve application speed in production.
 9. Security Best Practices
 - Protect against SQL injection, CSRF, XSS, and data breaches.
 - Implement HTTPS, secure cookies, and database encryption.
 10. Confidence to Build Production-Ready Apps
 - By the end, you'll be able to design, build, deploy, and maintain real-world backend systems from scratch.

Frequently Asked Questions (FAQs)

1. Who is this course for? This course is designed for beginners, intermediate learners, and professionals who want to master backend development using Django and PostgreSQL. Whether you're starting from scratch or switching from another tech stack (like MERN), this course will guide you from the fundamentals to building production-ready systems.

2. Do I need prior programming experience?

Basic programming knowledge is helpful but not required. We start with **Python fundamentals**, ensuring even complete beginners can follow along.

3. What will I be able to build after this course?

By the end of the course, you will be able to - Build fully functional web backends with Django - Design and optimize PostgreSQL databases - Implement authentication, APIs, and real-time features - Deploy applications to production - Optimize backend performance for scalability

4. Will this course cover only Django and PostgreSQL?

While the primary focus is on Django and PostgreSQL, you will also learn - REST API design - Server fundamentals - Deployment strategies (Docker, cloud hosting) - Performance tuning - Security best practices

5. How is this different from other backend courses?

Unlike most courses that only cover frameworks, this course - Teaches **backend fundamentals** that apply to any tech stack - Covers **real-world problems and solutions** - Guides you through **production grade deployment** - Provides **industry style assignments and projects**

6. Will I learn only theory or will there be projects?

You will work on - Mini projects for each module - A **capstone project** simulating a real-world web application - Real datasets and API integrations - Deployments to live servers

7. How will the live classes work?

- Classes will be interactive via video conferencing
- Recordings will be available for all sessions
- Q&A at the end of each class
- Practical coding exercises during live teaching

8. Will I get any certificate after completion?

Yes, you will receive a **certificate of completion** that can be added to your LinkedIn and resume.

9. Is there any post-course support?

Yes, - Access to a **community group** for ongoing discussions - Career guidance for backend developer roles - Code review sessions for your personal projects

10. Will there be an internship opportunity?

Yes. **High-performing students** from the course will be offered a **selective internship opportunity** to work on live projects and gain industry experience.

11. How will the internship help me?

The internship will allow you to, - Work in a real project environment. - Gain practical experience collaborating in a team. - Build a portfolio to showcase to future employers.